ML Experiment Number 10

Student Name : Sneha Gopal Bhalekar

Batch :cs1

Roll Number :cs05

Aim

To implement DBSCAN clustering using Python for customer segmentation on the Mall Customer Segmentation dataset and identify arbitrary-shaped clusters along with spatial outliers (noise).

Objectives

1. To understand the principles of Density-Based Spatial Clustering of Applications with Noise (DBSCAN).
2. To implement DBSCAN clustering using Python.3. To preprocess and feature-scale tabular customer data using StandardScaler.
3. To determine optimal hyperparameters (ε and $min_samples$) using k-distance graph analysis.
4. To identify core points, border points, and spatial outliers (noise points denoted by label -1).
5. To evaluate clustering performance using metrics like Silhouette Score.

Relevant Theory

###DBSCAN (Density-Based Spatial Clustering of Applications with Noise) DBSCAN is an unsupervised non-parametric clustering algorithm that groups data points based on local density estimations. Unlike centroid-based algorithms (e.g., K-Means), DBSCAN can discover arbitrary-shaped clusters and automatically isolate noisy data points without requiring the user to specify the total number of clusters in advance.

Core Concepts and Parameters:

1. ε (Epsilon): The maximum radius of the neighborhood around a given data point x.
2. $min_samples$ (MinPts): The minimum number of points within the ε-neighborhood required to form a dense region.

Point ClassificationsCore Point:

- A point that has at least $min_samples$ within its ε-neighborhood.
- Border Point: A point that is within the ε-neighborhood of a Core Point but has fewer than $min_samples$ in its own neighborhood.
- Noise Point (Outlier): A point that is neither a Core Point nor a Border Point. Noise points are assigned a cluster label of -1.

###Dataset Information Get the Mall Customer Segmentation dataset from Kaggle using the following link: https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python

The dataset contains spending patterns and demographic attributes of mall customers:

- CustomerID: Unique identifier for each customer.
- Gender: Male or Female.
- Age: Age of the customer.
- Annual Income (k$): Annual income of the customer in thousands of dollars.
- Spending Score (1-100): Score assigned based on customer behavior and purchasing patterns.

For clustering, we will use: Annual Income (k$) and Spending Score (1-100).

Task 1: Import Libraries and Load Dataset

- Load essential libraries (numpy, pandas, matplotlib, seaborn, sklearn).
- Load the Mall Customer Segmentation dataset (Mall_Customers.csv).

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score

# Load dataset
df = pd.read_csv("/content/Mall_Customers.csv")

# Display first five rows
print("First five rows of the dataset:")
print(df.head())

print("\nDataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())

print("\nMissing values:")
print(df.isnull().sum())
```

```
First five rows of the dataset:
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

Dataset shape: (200, 5)

Column names:
['CustomerID', 'Gender', 'Age', 'Annual Income (k$)', 'Spending Score
(1-100)']

Missing values:
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64
```

Task 2: Feature Selection and Standardization

- Select target features (Annual Income (k) and Spending Score (1-100)).
- Scale features using Standard Scaler to bring feature dimensions to a comparable range ($\mu=0, \sigma=1$), which is crucial for distance-based neighborhood computations.

```
# Select features for clustering
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("First five rows of scaled data:")
print(X_scaled[:5])

print("\nMean of scaled features:")
print(X_scaled.mean(axis=0))

print("\nStandard deviation of scaled features:")
print(X_scaled.std(axis=0))
```

```
First five rows of scaled data:
[[-1.73899919 -0.43480148]
 [-1.73899919  1.19570407]
 [-1.70082976 -1.71591298]
```

```
 [-1.70082976  1.04041783]
 [-1.66266033 -0.39597992]]

Mean of scaled features:
[-2.13162821e-16 -1.46549439e-16]

Standard deviation of scaled features:
[1. 1.]
```

Task 3: Hyperparameter Selection via k-Distance Graph

- Compute Nearest Neighbors using NearestNeighbors ($k = min_samples$).
- Plot sorted k-distances to locate the "elbow point," which indicates the optimal value for ε (Epsilon).

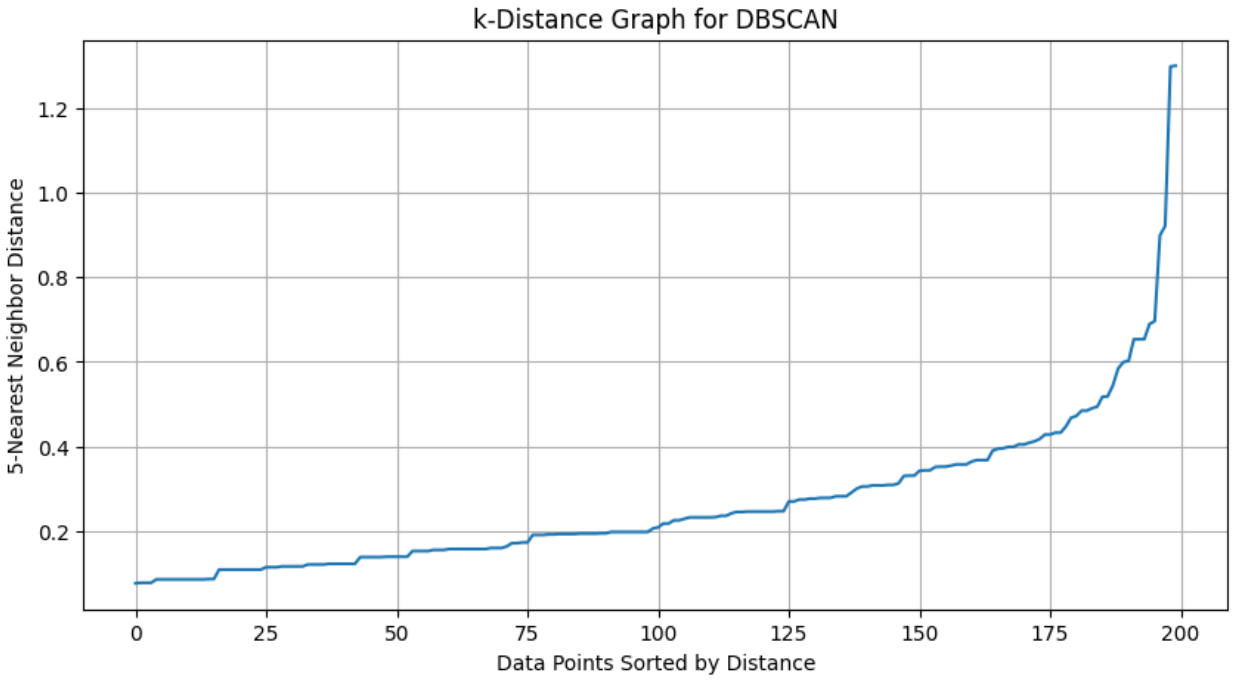
```
# Set min_samples
min_samples = 5

# Calculate nearest-neighbor distances
neighbors = NearestNeighbors(n_neighbors=min_samples)
neighbors.fit(X_scaled)

distances, indices = neighbors.kneighbors(X_scaled)

# Sort the k-distances
k_distances = np.sort(distances[:, min_samples - 1])

# Plot k-distance graph
plt.figure(figsize=(10, 5))
plt.plot(k_distances)
plt.xlabel("Data Points Sorted by Distance")
plt.ylabel(f"{min_samples}-Nearest Neighbor Distance")
plt.title("k-Distance Graph for DBSCAN")
plt.grid(True)
plt.show()
```



Task 4: Implement DBSCAN Clustering

- Instantiate sklearn.cluster.DBSCAN with the identified ε and $min_samples$.
- Fit the model on scaled data to generate cluster assignments and noise labels (-1).

```
# Initialize DBSCAN
dbscan = DBSCAN(eps=0.35, min_samples=5)

# Fit the model and obtain cluster labels
labels = dbscan.fit_predict(X_scaled)

# Display cluster labels
print("Cluster labels:")
print(labels)

print("\nUnique labels:", np.unique(labels))
```

```
Cluster labels:
[ 2  0  1  0  2  0  1 -1  1  0  1 -1  1  0  1  0  2  0  2 -1  2  0  1
0
  1  0  2  0  2  0  1  0  1 -1  1  0  1  0 -1  0  3 -1  3  3  3  3  3
3
  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3
3
  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3
3
  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3  3
```

```
3
  3  3  3  4  3  4  3  4  5  4  5  4  3  4  5  4  5  4  5  4  5  4  3
4
  5  4  3  4  5  4  5  4  5  4  5  4  5  4  5  4  3  4  5  4  5  4  5
4
  5 -1  5  4  5  4  5  4  5  4  5  4 -1  4  5  4 -1 -1 -1 -1 -1  4 -1
-1
 -1 -1 -1 -1 -1 -1 -1 -1]

Unique labels: [-1  0  1  2  3  4  5]
```

Task 5: Evaluate DBSCAN Clustering

- Cluster Count: Calculate the number of clusters generated (excluding noise points).
- Noise Point Analysis: Count and evaluate identified spatial outliers (label -1).
- Silhouette Score: Calculate Silhouette Score to evaluate cluster cohesion and separation.

```python
# Count clusters, excluding noise label -1
n_clusters = len(set(labels) - {-1})

# Count noise points
n_noise = np.sum(labels == -1)

print("Number of clusters:", n_clusters)
print("Number of noise points:", n_noise)
print("Total data points:", len(labels))
print("Noise percentage:", round(n_noise / len(labels) * 100, 2), "%")

# Calculate silhouette score for non-noise points
non_noise_mask = labels != -1
non_noise_labels = labels[non_noise_mask]
non_noise_data = X_scaled[non_noise_mask]

if len(set(non_noise_labels)) >= 2:
    score = silhouette_score(non_noise_data, non_noise_labels)
    print("Silhouette Score:", round(score, 4))
else:
    print("Silhouette Score cannot be calculated: fewer than 2
clusters.")

# Count clusters, excluding noise label -1
n_clusters = len(set(labels) - {-1})

# Count noise points
n_noise = np.sum(labels == -1)

print("Number of clusters:", n_clusters)
print("Number of noise points:", n_noise)
```

```
print("Total data points:", len(labels))
print("Noise percentage:", round(n_noise / len(labels) * 100, 2), "%")

# Calculate silhouette score for non-noise points
non_noise_mask = labels != -1
non_noise_labels = labels[non_noise_mask]
non_noise_data = X_scaled[non_noise_mask]

if len(set(non_noise_labels)) >= 2:
    score = silhouette_score(non_noise_data, non_noise_labels)
    print("Silhouette Score:", round(score, 4))
else:
    print("Silhouette Score cannot be calculated: fewer than 2
clusters.")

# Count clusters, excluding noise label -1
n_clusters = len(set(labels) - {-1})

# Count noise points
n_noise = np.sum(labels == -1)

print("Number of clusters:", n_clusters)
print("Number of noise points:", n_noise)
print("Total data points:", len(labels))
print("Noise percentage:", round(n_noise / len(labels) * 100, 2), "%")

# Calculate silhouette score for non-noise points
non_noise_mask = labels != -1
non_noise_labels = labels[non_noise_mask]
non_noise_data = X_scaled[non_noise_mask]

if len(set(non_noise_labels)) >= 2:
    score = silhouette_score(non_noise_data, non_noise_labels)
    print("Silhouette Score:", round(score, 4))
else:
    print("Silhouette Score cannot be calculated: fewer than 2
clusters.")
```

```
Number of clusters: 6
Number of noise points: 23
Total data points: 200
Noise percentage: 11.5 %
Silhouette Score: 0.5577
Number of clusters: 6
Number of noise points: 23
Total data points: 200
Noise percentage: 11.5 %
Silhouette Score: 0.5577
Number of clusters: 6
Number of noise points: 23
```

```
Total data points: 200
Noise percentage: 11.5 %
Silhouette Score: 0.5577
```

Task 6: Analyze Results and Cluster Statistics

- Extract cluster labels and append them back to the original dataset.
- Calculate mean statistics (e.g., average Annual Income and Spending Score) per cluster to interpret final customer segments and evaluate detected noise points.

```
# Add cluster labels to the original dataset
df['Cluster'] = labels

# Calculate cluster statistics, excluding noise
cluster_stats = df[df['Cluster'] != -1].groupby('Cluster').agg(
    Customer_Count=('CustomerID', 'count'),
    Average_Age=('Age', 'mean'),
    Average_Income=('Annual Income (k$)', 'mean'),
    Average_Spending_Score=('Spending Score (1-100)', 'mean')
).round(2)

print("Cluster Statistics:")
print(cluster_stats)

# Display noise points
noise_points = df[df['Cluster'] == -1]

print("\nNumber of noise points:", len(noise_points))
print("\nNoise Points:")
print(noise_points.head(10))

# Visualize clusters and noise points
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='Cluster',
    palette='tab10',
    style='Cluster',
    s=80
)

plt.title("DBSCAN Customer Segmentation")
plt.xlabel("Annual Income (k$)")
```

```
plt.ylabel("Spending Score (1-100)")
plt.legend(title="Cluster", bbox_to_anchor=(1.05, 1), loc='upper
left')
plt.tight_layout()
plt.show()
```

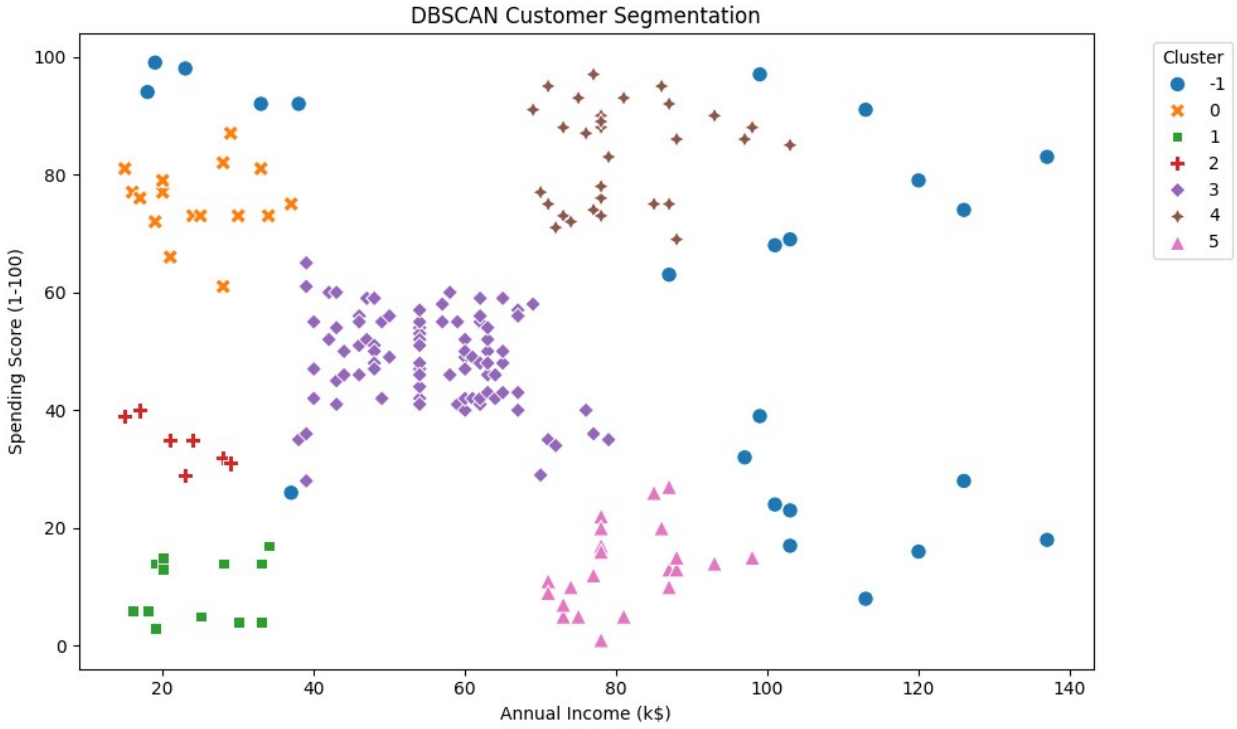
```
Cluster Statistics:
         Customer_Count  Average_Age  Average_Income
Average_Spending_Score
Cluster

0                    16        24.81           24.75
75.38
1                    12        48.75           24.58
9.58
2                     7        36.71           22.43
34.43
3                    88        42.88           55.23
48.58
4                    31        32.84           80.29
83.19
5                    23        41.22           80.96
12.78

Number of noise points: 23

Noise Points:
     CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-
100)  \
7             8  Female   23                  18
94
11           12  Female   35                  19
99
19           20  Female   35                  23
98
33           34    Male   18                  33
92
38           39  Female   36                  37
26
41           42    Male   24                  38
92
169         170    Male   32                  87
63
180         181  Female   37                  97
32
184         185  Female   41                  99
39
185         186    Male   30                  99
97
```

	Cluster
7	-1
11	-1
19	-1
33	-1
38	-1
41	-1
169	-1
180	-1
184	-1
185	-1



Conclusion

The experiment successfully implemented DBSCAN clustering on the Mall Customer Segmentation dataset. Feature standardization ensured balanced neighborhood distance calculations across income and spending metrics. By analyzing k-distance graphs, optimal ε and $min_samples$ parameters were selected, enabling the discovery of density-defined customer segments and explicit identification of spatial noise points (outliers) without requiring a pre-defined cluster count.